System of Work Assistant

Product Requirements Document

v0.1 Draft - 2026-06-27

A Mac-first, open-source, self-hosted assistant for personal life, side projects, and employer work - built on an Obsidian-compatible Markdown brain, GBrain, Temporal, Hermes, and Claude Agent SDK.

Document purpose: define the product, architecture, requirements, boundaries, risks, and implementation plan for the first product-quality single-user release.

1. Document Control

Field	Value
Document	System of Work Assistant PRD
Version	v0.1 Draft
Primary user model	Single user, product-quality, open-source installable later
Primary platform	Mac-first desktop application
Canonical semantic knowledge	Obsidian-compatible Markdown Git repositories
Knowledge engine	GBrain required in V1
Agent runtimes	Hermes Agent adapter and Claude Agent SDK adapter in V1
Workflow engine	Temporal for cross-system workflows
Status	Draft for architectural review and implementation planning

1.1 Decision Summary

Decision	Direction
Product posture	Single-user first, open-source/self-hosted, product-quality architecture.
Workspace model	Three default logical workspaces: Employer Work, Personal Business / Side Projects, Personal Life, plus a Global Coordination Layer.
Knowledge store	Markdown Git repos are canonical; Obsidian opens them as human-facing workspaces.
GBrain	Required in V1 for full knowledge capabilities: search, graph, schema, Minions, dream, MCP, health, synthesis. Durable semantic writes route through the custom layer.
Runtimes	Both Hermes and Claude Agent SDK are V1 runtime adapters behind AgentRuntimePort.
Workflow durability	Temporal owns cross-system workflows, approval waits, external writes, and retries.
GBrain jobs	GBrain Minions own GBrain-internal maintenance, indexing, dream, extraction, and embed jobs.
NotebookLM	Run API spike; fallback is Drive-backed managed source documents.
External systems	Calendar, Todoist, Linear, Asana, Granola, Telegram, Google Drive/Docs, GitHub, YouTube, Podcast/RSS are first-class V1 or V1-adjacent targets.

2. Table of Contents

3. Executive Summary

The System of Work Assistant is a Mac-first, open-source, self-hosted personal operating system for managing employer work, side projects, and personal life. It combines a human-readable Markdown knowledge base with a strong AI knowledge engine, durable workflows, controlled agent runtimes, and integrations with the tools where operational work already happens.

The product is not a generic chatbot and not a replacement for Calendar, Todoist, Linear, Asana, Granola, NotebookLM, or Obsidian. It is an orchestration and memory layer that understands how those systems relate to projects, meetings, tasks, decisions, sources, and reviews.

> **North star: every meaningful meeting, source, decision, task, and project update should land in the correct workspace, update the correct project memory, route the right operational actions, and remain inspectable in user-owned Markdown.**

3.1 Core Thesis

- The assistant should be single-user and self-hosted in V1, but architected so other users can install it from GitHub later.
- The canonical semantic memory is an Obsidian-compatible Markdown Git repository, not a hidden app database.
- GBrain is required in V1 as the knowledge engine for retrieval, graph, schema, health, Minions, dream cycle, and MCP access.
- Hermes Agent and Claude Agent SDK are both runtime adapters in V1; neither is the source of truth.
- Temporal owns cross-system workflows, approvals, retries, and external writes.
- A custom KnowledgeWriter is the only semantic Markdown writer.
- Separate workspaces protect data boundaries; a Global Coordination Layer enables scheduling and priority intelligence across them.

3.2 What This Product Must Do

Scenario	Expected Assistant Behavior
Granola meeting about a side project	Identify the Personal Business workspace and project, create/update the meeting note, update the project, extract explicit tasks, propose calendar follow-ups, and refresh the dashboard.
Doctor appointment request	Route to Personal Life, check global availability across personal/business/work calendars, create a personal calendar event or task according to policy.
Employer work meeting	Route to Employer Work, keep raw details inside that workspace, update the work project and people notes, route tasks to Linear or Asana.
YouTube or podcast source	Use source-specific extraction, classify workspace/project, create a research note, update relevant project/concept pages, and sync to NotebookLM if mapped.
Implementation project	Parse IMPLEMENTATION_PLAN.md and/or Linear project status, compute deterministic progress, summarize blockers and next actions.
Morning brief	Produce one global brief with workspace-specific sections, conflicts, priorities, overdue work, and suggested plan without leaking sensitive raw data across workspaces.

4. Product Vision, Principles, and Goals

4.1 Vision

Create an always-on assistant that keeps the user oriented across work, side projects, and personal life. The assistant should know what projects exist, what was decided, what commitments were made, what sources matter, which tasks are critical, which meetings are upcoming, and how to avoid conflicts across workspaces.

4.2 Product Principles

Principle	Meaning
User-owned memory	Long-term semantic knowledge lives in Markdown Git repositories that can be opened in Obsidian.
One writer	All durable semantic note changes flow through KnowledgeWriter. No competing autonomous file writers.
External systems stay authoritative	Calendar owns time, Todoist owns personal task completion, Linear/Asana own collaborative work status, Granola owns raw meeting transcripts.
Workspace-aware by default	Every event, source, task, and note belongs to a workspace before it belongs to a project.
Global coordination without leakage	Availability and priorities can be coordinated across workspaces without exposing raw confidential content.
Runtime-neutral execution	Hermes and Claude Agent SDK are adapters. Workflows do not depend on one specific agent runtime.
Deterministic before latent	Use code for identity, routing, counting, idempotency, policy, and progress. Use agents for judgment, synthesis, and explanation.
Approvals are product state	Pending actions are persisted, auditable, and visible in Mac and Telegram interfaces.
Open-source installability	V1 is built for one user, but with clean install, configuration, and documentation paths for later users.

4.3 Goals

- Provide a unified daily operating system for employer work, side projects, and personal life.
- Automatically route meeting transcripts, captured thoughts, files, links, videos, podcasts, tasks, and calendar requests to the correct workspace and project.
- Maintain high-quality project memory, decisions, people context, meeting history, sources, and reviews.
- Offer a sleek Mac dashboard and Copilot interface for briefs, tasks, project health, calendar, approvals, ingestion, and recent changes.
- Use Telegram as a remote interface for capture, questions, approvals, and notifications.
- Preserve user ownership and inspectability through Obsidian-compatible Markdown.
- Use GBrain for advanced search, typed graph, schema, synthesis, Minions, dream cycle, and health.
- Support both Hermes and Claude Agent SDK as V1 runtime adapters.

4.4 Non-Goals for V1

- No SaaS multi-tenant billing, organization administration, or team onboarding in V1.
- No autonomous email or external messaging sends in V1.
- No unsupported browser automation against NotebookLM unless a validated spike proves it safe and reliable.
- No attempt to replace Todoist, Linear, Asana, Google Calendar, Granola, Obsidian, or NotebookLM.

- No arbitrary direct writes by Hermes, Claude, GBrain dream jobs, or external MCP clients into the Markdown brain.
- No model-only project progress percentages; progress must derive from structured providers or deterministic evidence.
- No single global unscoped memory search across employer work and personal data by default.

5. Users, Use Cases, and Success Metrics

5.1 Primary User

The V1 user is a single power user who manages employer work, personal business/side projects, and personal life. The user is comfortable with developer-style installation and values ownership, extensibility, automation, and transparency over consumer-app simplicity.

5.2 Future Users

The project should be installable by other technically capable users from GitHub. V1 should therefore include configuration presets and clear deployment paths, even if the initial product is not packaged as a commercial SaaS or App Store app.

5.3 Core Use Cases

Use Case	Description	V1 Priority
Meeting closeout	Turn a Granola transcript into meeting notes, decisions, project updates, tasks, calendar proposals, and people context.	P0
Daily brief	Unify calendars, tasks, project status, critical deadlines, waiting items, and suggested plan across workspaces.	P0
Project dashboard	Show project health, progress, next actions, blockers, recent changes, and linked sources.	P0
Task routing	Route commitments to Todoist, Linear, or Asana based on workspace, project, and task type.	P0
Calendar coordination	Create/suggest events while checking conflicts across personal, business, and employer contexts.	P0
Ingestion inbox	Capture links, files, voice, YouTube, podcasts, PDFs, and raw thoughts with reviewable routing.	P1
NotebookLM sync	Maintain project-specific source packs in NotebookLM via API if validated, otherwise Drive-managed docs.	P1
Weekly/monthly review	Summarize progress, decisions, recurring blockers, commitments, and project adjustments.	P1
Implementation plan progress	Parse IMPLEMENTATION_PLAN.md and external PM status for technical projects.	P1
Open-source install	Provide docs and scripts so other users can run the system locally/self-hosted.	P1

5.4 Success Metrics

Metric	Target for V1
Meeting closeout accuracy	90%+ of processed meetings routed to the correct workspace and project in test corpus.
Duplicate prevention	0 duplicate external tasks/calendar events in replay/retry tests.

Knowledge write reliability	100% of approved semantic writes are present in Markdown or visible in retry/error outbox.
Calendar conflict safety	100% of scheduling proposals check all configured availability sources.
Approval visibility	100% of pending external side effects appear in Mac app and/or Telegram approval inbox.
Retrieval usefulness	GBrain search/think returns relevant project/person/decision context for 90%+ of benchmark queries.
Workspace isolation	0 raw employer-work documents surface in personal workspace outputs without explicit permission.
Install reproducibility	Fresh install path succeeds on a clean Mac dev environment with documented prerequisites.

6. Workspace and Privacy Model

Workspace boundaries follow ownership, confidentiality, and default tool routing - not simply whether something feels like work. Side projects are work-like but personally owned, so they should not be mixed with employer work.

6.1 Default Workspaces

Workspace	Purpose	Default Tools	Default Visibility
Employer Work	Official job/company projects, meetings, tasks, people, decisions, and confidential work data.	Work calendar, Granola work account, Linear, Asana, work NotebookLM/Drive if approved.	Metadata-only outside workspace by default.
Personal Business / Side Projects	Income-generating side projects, indie software, consulting, content, clients, and personally owned professional work.	Personal calendar, Todoist, personal Linear/Asana if configured, Granola personal, NotebookLM/Drive.	Sanitized summaries may coordinate globally.
Personal Life	Health, home, finance, family, relationships, learning, admin, and personal goals.	Personal calendar, Todoist, optional Drive/NotebookLM.	Metadata-only or sanitized summaries by default.
Global Coordination Layer	Not a normal workspace. Stores availability, conflict metadata, cross-workspace priorities, identity map, and sanitized summaries.	Reads coordination metadata from all workspaces.	Visible to assistant for planning but not a raw content store.

6.2 Workspace versus Project versus Area

Concept	Definition	Examples
Workspace	Trust, ownership, routing, and policy boundary.	Employer Work; Personal Business; Personal Life.
Area	Ongoing responsibility inside a workspace with no finish line.	Health, Finance, Product, Consulting, Learning.
Project	Finite outcome with status, meetings, tasks, decisions, sources, and progress.	Launch Atlas beta; File taxes; Prepare Q3 roadmap.
Task	Actionable commitment owned by a person/system, often managed in Todoist, Linear, or Asana.	Send Maya architecture diagram by Friday.
Source	Imported information artifact that may update knowledge.	Granola transcript, PDF, YouTube video, podcast, web article.

6.3 Cross-Workspace Visibility Levels

Level	Name	Allowed Cross-Workspace Use
0	Isolated	No cross-workspace access. Use for journals, sensitive work strategy, medical details, secrets.
1	Coordination metadata	Busy/free blocks, due dates, generic priority labels, non-sensitive deadlines.

2	Sanitized summary	High-level summary without raw details, such as "critical work deadline this week".
3	Explicit link	User-approved relationship between projects, people, or sources across workspaces.
4	Full access	Explicitly allowed for a specific user request or workflow; never default for employer work.

6.4 Cross-Workspace Scheduling Requirement

The assistant must be able to schedule personal events while respecting employer-work and side-project commitments without exposing raw work content.

Example:

User: "Find a time for a doctor appointment next week."

Route:

- Workspace: Personal Life
- Area: Health
- Calendar destination: personal calendar

Global coordination reads:

- Personal calendar busy/free
- Personal Business focus blocks and deadlines
- Employer Work busy/free only, not meeting transcripts

Result:

- Suggest available windows
- Explain conflicts generically
- Create private event automatically if policy allows

7. System Architecture

7.1 High-Level Architecture

User Interfaces

Mac App | Telegram | Obsidian | Webhooks | Schedules

|

v

Assistant Control Plane

Events | Temporal Workflows | Policy | Approvals | Audit | Routing

|

v

Agent Runtime Broker

HermesRuntimeAdapter | ClaudeAgentSdkRuntimeAdapter | Deterministic Workers

|

v

Deterministic Application Layer

KnowledgeWriter | Tool Gateway | Connector Gateway | Project Registry

| |

v v

Markdown Knowledge Repos External Systems

Opened in Obsidian Calendar | Todoist | Linear | Asana | Granola

Indexed by GBrain Drive | NotebookLM | GitHub | Telegram

|

v

GBrain Knowledge Engine

Search | Think | Graph | Schema | Minions | Dream | Health | MCP

7.2 Component Responsibilities

Component	Owns	Does Not Own
Assistant Control Plane	Event ingress, Temporal workflows, policy, approvals, idempotency, routing, audit, notifications, dashboard read models.	Semantic knowledge truth or raw external records.
Agent Runtime Broker	Runtime selection, job envelope, capability matching, cost/time limits, result validation.	Model reasoning itself or durable state.
Hermes Adapter	Hermes agent execution, messaging gateway, MCP-heavy work, subagents, runtime-local capabilities.	Cross-system workflow truth or unrestricted writes.
Claude Agent SDK Adapter	First-party Claude runtime execution, structured outputs, tool permissions, hooks, subagents.	Global orchestration or direct external writes outside policy.
KnowledgeWriter	Validated semantic Markdown mutations, conflict handling, revision recording, write-through, GBrain sync trigger.	External task/calendar state.
GBrain	Retrieval, graph, schema, MCP, Minions, dream cycle, health, derived index, semantic query/synthesis.	Final approval policy or exclusive semantic truth.
Obsidian	Human-facing Markdown editing, browsing, graph, visual inspection, manual correction.	Runtime, scheduler, queue, integrations.
Temporal	Cross-system workflows, retries, approval waits, external-side-effect orchestration.	GBrain-internal indexing jobs.
GBrain Minions	GBrain-internal jobs: sync, embed, extract, dream, health.	Calendar/Todoist/Linear/Asana write orchestration.

7.3 Ports and Adapters

Core ports:

- AgentRuntimePort
- KnowledgePort
- KnowledgeWriterPort
- CalendarPort
- TaskPort
- MeetingTranscriptPort
- NotebookPort
- ApprovalPort

- NotificationPort
- ProjectRegistryPort

V1 adapters:

- HermesRuntimeAdapter
- ClaudeAgentSdkRuntimeAdapter
- GBrainAdapter
- MarkdownKnowledgeWriter
- GoogleCalendarAdapter
- TodoistAdapter
- LinearAdapter
- AsanaAdapter
- GranolaAdapter
- NotebookLMDriveAdapter or NotebookLMApiAdapter after spike
- TelegramAdapter
- GitHubAdapter

7.4 Runtime Decision

The system will build Option B foundation from day one: a control plane above replaceable runtimes. Hermes is not the orchestrator; it is one agent runtime adapter. Claude Agent SDK is also a V1 adapter. The product avoids a custom low-level model loop in V1, but builds the runtime-neutral broker and governance layer required to add or replace runtimes later.

8. Product Surfaces and User Experience

8.1 Mac Desktop App

The Mac app is the primary human interface. It should be built with Tauri, React, and TypeScript. Initial distribution can be direct GitHub install/build, with signed/notarized builds later if useful.

Surface	Purpose
Global Today Dashboard	Unified calendar, top priorities, conflicts, overdue/critical items, waiting items, suggested plan.
Workspace Tabs	Filter views by Employer Work, Personal Business, Personal Life, or All.
Project Dashboard	Project health, deterministic progress, blockers, next actions, recent changes, meetings, decisions, sources.
Copilot	Ask questions, run workflows, explain project state, process sources, prepare meetings.
Ingestion Inbox	Review sources from Telegram, files, URLs, YouTube, podcasts, PDFs, Granola, Drive, GitHub.
Approval Inbox	Approve, edit, reject, or defer external actions and sensitive knowledge mutations.
Calendar View	Unified availability, workspace filters, conflicts, focus blocks, follow-ups.
Recent Changes	Audit timeline for notes, projects, tasks, calendar events, sources, jobs, approvals.
System Health	Connector status, GBrain health, workflow failures, queue depth, failed writes, cost, sync lag.

8.2 Telegram

Telegram is a remote interaction and approval channel. It supports capture, quick questions, brief delivery, meeting preparation, approval buttons, and status notifications. Webhook mode is preferred for always-on delivery; polling is acceptable for local development.

- Text capture and commands.
- Voice memo capture and transcription pipeline.
- Photo/document/link ingestion.
- Approval cards with inline buttons.
- Brief notifications and workflow failure alerts.
- Strict sender allowlist and workspace-aware routing.

8.3 Obsidian

Obsidian remains a first-class user surface. It opens the Markdown repositories, allowing the user to inspect and edit what the assistant believes. Obsidian is not the integration runtime or scheduler.

- Editable project, meeting, decision, person, daily, review, source, and research notes.
- Human-owned sections that the assistant must not overwrite.
- Assistant-managed sections bounded by explicit markers.
- Optional Bases/views for projects, meetings, tasks, reviews, and sources.
- Graph visualization and manual correction.

9. Functional Requirements

9.1 Workspace and Identity Requirements

ID	Requirement	Priority
WS-1	System shall support at least three logical workspaces: Employer Work, Personal Business, Personal Life.	P0
WS-2	Every event, source, meeting, task, note, and project shall be assigned a workspace before durable processing.	P0
WS-3	System shall maintain global coordination metadata for availability, deadlines, cross-workspace priorities, and identity matching.	P0
WS-4	System shall prevent raw Employer Work content from appearing in other workspaces by default.	P0
WS-5	System shall support user-approved explicit cross-workspace links.	P1
WS-6	Open-source users shall be able to choose Simple, Professional, Founder/Side Project, or Advanced workspace presets.	P1

9.2 Knowledge and Retrieval Requirements

ID	Requirement	Priority
KN-1	GBrain shall be required in V1 as the knowledge engine.	P0
KN-2	GBrain shall provide search, think/synthesis, graph traversal, timelines, schema, health, MCP, Minions, and dream-cycle capabilities.	P0

KN-3	Semantic knowledge shall be represented in Markdown Git repositories that can be opened in Obsidian.	P0
KN-4	KnowledgeWriter shall be the only direct writer for durable semantic Markdown changes.	P0
KN-5	If GBrain or an agent proposes a semantic change, it shall submit a KnowledgeMutationPlan rather than writing directly.	P0
KN-6	Every approved semantic change shall be auditable and linked to source evidence where available.	P0
KN-7	Human-owned note sections shall not be overwritten by the assistant.	P0
KN-8	Assistant-generated note sections shall use explicit start/end markers and stable IDs.	P1

9.3 Runtime and Workflow Requirements

ID	Requirement	Priority
RT-1	System shall expose an AgentRuntimePort with Hermes and Claude Agent SDK adapters in V1.	P0
RT-2	Agent jobs shall specify workspace, capability, context references, tool policy, max runtime, max cost, output schema, and idempotency key.	P0
RT-3	Agent results shall be validated against schema before side effects occur.	P0
RT-4	Temporal shall orchestrate cross-system workflows and approval waits.	P0
RT-5	GBrain Minions shall be used for GBrain-internal jobs only unless explicitly bridged through control-plane policy.	P0
RT-6	Hermes cron and Kanban may be supported through the Hermes adapter but shall not replace Temporal as the system workflow source of truth.	P1

9.4 Meeting Requirements

ID	Requirement	Priority
MTG-1	System shall ingest Granola meetings and correlate them to calendar events, workspaces, projects, attendees, and prior meeting history.	P0
MTG-2	System shall create or update one canonical meeting note per meeting.	P0
MTG-3	System shall extract explicit decisions, explicit commitments, risks, open questions, follow-ups, and project state changes.	P0
MTG-4	System shall never infer task owners or due dates when not stated; unknowns shall be marked TBD or routed for clarification.	P0
MTG-5	System shall update project notes, person notes, decision records, and daily notes through KnowledgeWriter.	P0
MTG-6	System shall route task proposals to Todoist, Linear, or Asana according to workspace/project policy.	P0
MTG-7	System shall propose calendar follow-ups and apply low-risk writes automatically only when policy allows.	P0

9.5 Project and Task Requirements

ID	Requirement	Priority
PRJ-1	All projects shall share a common project model across workspaces.	P0
PRJ-2	Projects shall have stable IDs, workspace IDs, aliases, status, outcome, owner, task system mapping, calendar mapping, source mapping, and progress providers.	P0
PRJ-3	Technical projects shall support IMPLEMENTATION_PLAN.md progress providers.	P1
PRJ-4	Projects shall display deterministic progress derived from providers, not invented model percentages.	P0
TASK-1	Personal-life and personal-business tasks shall route to Todoist by default unless project policy overrides.	P0
TASK-2	Technical work tasks shall route to Linear when configured.	P0
TASK-3	Operations/general work tasks shall route to Asana when configured.	P0
TASK-4	Tasks assigned to other people or shared systems shall require approval unless explicitly configured otherwise.	P0

9.6 Ingestion Requirements

ID	Requirement	Priority
ING-1	All sources shall enter through a canonical ingestion event with source ID, workspace, content hash, origin, content type, user tags, sensitivity, and routing hints.	P0
ING-2	System shall support Granola transcripts, Telegram messages/files/voice, desktop file upload, URLs, YouTube, podcasts/RSS, PDFs, images/OCR, web articles, GitHub/code sources, and Drive docs.	P1
ING-3	YouTube and podcast extraction behavior from Obsidian Second Brain shall be ported into source adapters.	P1
ING-4	Low-confidence routing shall remain in the ingestion inbox rather than being guessed.	P0
ING-5	Imported content shall be treated as untrusted and shall not be allowed to issue instructions to agents or tools.	P0
ING-6	The system shall deduplicate by content hash, external ID, source URI, and workflow idempotency key.	P0

9.7 Calendar, NotebookLM, and Brief Requirements

ID	Requirement	Priority
CAL-1	System shall read all configured calendars for global availability and conflict detection.	P0
CAL-2	Private focus blocks and non-shared reminders may be created automatically when policy allows.	P0
CAL-3	Inviting others, changing shared meetings, or cancelling meetings shall require approval.	P0
NLM-1	NotebookLM API spike shall determine whether direct notebook/source management is possible.	P0
NLM-2	If API is not viable, system shall sync project notebooks through managed Google Drive docs/sheets/markdown exports.	P0

BRF-1	System shall generate global and workspace-specific daily, weekly, and monthly briefs.	P0
BRF-2	Briefs shall be saved to Markdown and displayed in the Mac dashboard.	P0
BRF-3	Briefs shall cite or link supporting notes, tasks, calendar events, and sources where practical.	P1

10. Data Model and Canonical Objects

10.1 Source of Truth Matrix

Data	Authoritative System	Representation in Assistant
Scheduled events	Google Calendar	Calendar snapshot, meeting note link, project/date references.
Personal task status	Todoist	Task reference, project next action, commitment summary.
Technical work execution	Linear	External task/project link, project progress provider.
General operations work	Asana	External task/project link, project status provider.
Raw meeting transcript	Granola	Transcript link/source record, meeting synthesis, decisions, commitments.
Semantic project knowledge	Markdown Git repository	Project notes, decisions, meeting summaries, sources, reviews.
Search/graph/index	GBrain derived database	Rebuildable from Markdown and approved external records.
Workflow state	Control plane / Temporal	Operational state only; not semantic memory.
Notebook analysis	NotebookLM / Drive	Managed source docs and returned insights ingested as sources.

10.2 Core Object Sketches

```
Workspace {
  id: string
  name: string
  type: employer_work | personal_business | personal_life
  dataOwner: employer | user | client
  defaultVisibility: isolated | coordination | sanitized | full
  gbrainBrainId?: string
  gbrainSourceId?: string
  calendars: CalendarAccount[]
  taskSystems: TaskSystemMapping
  notebookPolicy: NotebookPolicy
}

Project {
  id: string
  workspaceId: string
  name: string
```

```
  aliases: string[]
  kind: employer_project | side_income | client | life_admin | learning | code
  status: active | paused | completed | archived | someday
  outcome: string
  definitionOfDone?: string
  taskSystem: todoist | linear | asana | none
  externalProjectRefs: ExternalRef[]
  calendarAliases: string[]
  notebookMappings: NotebookMapping[]
  progressProviders: ProgressProvider[]
}

KnowledgeMutationPlan {
  planId: string
  workspaceId: string
  sourceRefs: SourceRef[]
  creates: NoteCreate[]
  patches: NotePatch[]
  linkMutations: LinkMutation[]
  frontmatterUpdates: FrontmatterPatch[]
  externalActionProposals: ProposedAction[]
  confidence: number
  requiresApproval: boolean
}
```

10.3 Project Frontmatter Example

```
---
assistant-id: project_atlas
workspace-id: personal-business
type: project
project-kind: side-income
status: active
owner: user
outcome: "Launch the Mac-first system-of-work assistant MVP"

task-system: linear-personal
linear-project-id: lin_proj_123
asana-project-id: null
todoist-project-id: todoist_456
```

```
calendar-aliases:
  - Atlas
  - System of Work Assistant

granola-folder-id: granola_atlas
notebooklm-key: atlas-research
google-drive-folder-id: drive_folder_789
repository-path: ~/Projects/system-of-work-assistant
implementation-plan: IMPLEMENTATION_PLAN.md

cross-workspace-visibility: sanitized
---
```

11. Core Workflows

11.1 Meeting Closeout Workflow

1. Granola transcript completed event enters the control plane.
2. Workflow correlates transcript to calendar event, workspace, project, and attendees.
3. Workflow gathers GBrain context for project, people, prior meetings, decisions, open tasks, and relevant sources.
4. AgentRuntimeBroker selects Hermes or Claude Agent SDK according to job policy.
5. Agent returns structured MeetingCloseResult with decisions, commitments, project updates, people updates, risks, open questions, and proposed external actions.
6. Validator rejects unsupported claims, inferred owners, missing evidence, and ambiguous project routing.
7. KnowledgeWriter updates meeting, project, person, decision, daily, and source notes.
8. Tool Gateway creates low-risk external actions or requests approval for sensitive/shared changes.
9. GBrain sync/index/dream jobs run as appropriate.
10. Dashboard and Telegram summarize the result.

Acceptance: replaying the same meeting event must not create duplicate meeting notes, tasks, or calendar events.

11.2 Scheduling a Personal Appointment

11. User asks via Mac Copilot or Telegram to schedule an appointment.
12. Intent router assigns Personal Life workspace and Health area unless user specifies otherwise.
13. Global Coordinator fetches busy/free metadata across Personal Life, Personal Business, and Employer Work calendars.
14. Assistant proposes windows that avoid conflicts and respect buffers.

15. If event has no invitees and policy allows, system creates event automatically in the personal calendar.
16. If invitees, external notifications, or sensitive details are involved, system requests approval.
17. Daily note and relevant area/project note are updated if the appointment affects commitments or plans.

11.3 Source Ingestion Workflow

18. Source arrives from desktop, Telegram, URL, YouTube, podcast, Drive, GitHub, Granola, or watched folder.
19. Ingestion service registers SourceEnvelope, validates content type, computes hash, and applies safety policy.
20. Source-specific adapter extracts content and metadata.
21. Router classifies workspace, project, tags, sensitivity, and desired processing depth.
22. Low-confidence items remain in Ingestion Inbox.
23. Agent extracts knowledge delta: summary, claims, decisions, commitments, entities, dates, contradictions, links.
24. KnowledgeWriter applies approved note mutations.
25. Optional NotebookLM sync updates project-managed source documents.
26. GBrain indexes and graph-links the new/changed knowledge.

11.4 Daily Brief Workflow

27. Temporal schedule fires daily at user-configured time.
28. Connector sync refreshes calendars, Todoist, Linear, Asana, and recent Granola events.
29. GBrain retrieves active projects, waiting items, critical facts, recent changes, and relevant timeline entries.
30. Briefing agent creates a global brief with workspace sections and citations/links.
31. Brief is written to Markdown and dashboard read model.
32. Telegram sends a short summary and link to dashboard.

11.5 Project Sync Workflow

33. Workflow runs on demand, nightly, or after relevant events.
34. Project registry resolves external task systems and progress providers.
35. Deterministic progress parser reads IMPLEMENTATION_PLAN.md checkboxes or external project status.
36. Agent synthesizes explanation of progress, blockers, risks, waiting items, and next actions.
37. KnowledgeWriter updates project current status and dashboard projection.
38. External writes are proposed only when needed and governed by approval policy.

12. Integration Requirements

12.1 Required V1 Integrations

Integration	Role	Default Write Policy
GBrain	Knowledge engine, retrieval, graph, MCP, Minions, dream, health.	Internal maintenance allowed; semantic writes through KnowledgeWriter.

Obsidian Markdown Repo	Human-facing knowledge workspace.	KnowledgeWriter only, plus user manual edits.
Hermes Agent	Runtime adapter, messaging/gateway, MCP-heavy execution, subagents.	No direct durable semantic or external writes without gateway.
Claude Agent SDK	Runtime adapter for structured Claude jobs.	Tool permissions and control-plane approvals.
Google Calendar	Scheduled commitments and availability.	Private user-only writes automatic; shared/invite writes approval.
Granola	Meeting transcript source.	Read/source only in V1.
Todoist	Personal tasks.	Self-owned task creation may be automatic.
Linear	Technical project execution.	Self-assigned task creation may be automatic; shared changes approval.
Asana	General operations project execution.	Shared task/status changes approval.
Telegram	Remote interface and approvals.	No external system changes except approval commands.
Google Drive/Docs	NotebookLM managed docs and shared source files.	Managed doc updates via policy.
NotebookLM	Project-specific analysis workspace.	API spike; fallback is Drive-managed source docs.
GitHub	Source repo context, open-source install, code project linking.	Repo writes approval unless user explicitly works in code mode.
YouTube/Podcast/RSS	Research/source ingestion.	Read/extract only; knowledge writes through KnowledgeWriter.

12.2 NotebookLM Approach

NotebookLM direct API support remains an explicit spike. If a supported public API can create/manage notebooks and sources, V1 may include NotebookLMApiAdapter. If not, V1 uses Drive-backed sync: the assistant creates and updates managed Drive docs that the user adds to NotebookLM notebooks.

Drive-backed NotebookLM fallback:

Project Atlas

-> 00 Project Brief Google Doc

-> 01 Decision Log Google Doc

-> 02 Meeting Digest Google Doc

-> 03 Research and Sources Google Doc

-> 04 Open Questions Google Doc

-> User adds these docs to NotebookLM once

-> Assistant updates Docs; NotebookLM auto-syncs Drive sources where supported

13. Agent Runtime Strategy

13.1 Runtime-Neutral Contract

```
AgentJob {
  id: string
  workflowRunId: string
  workspaceId: string
  capability: meeting.close | daily.brief | project.sync | source.ingest | task.route | notebooklm.sync
  prompt: string
  contextRefs: ContextRef[]
  outputSchema: JsonSchema
  toolPolicy: ToolPolicy
```

```
  maxRuntimeSeconds: number
  maxCostUsd?: number
  idempotencyKey: string
}

AgentResult {
  jobId: string
  runtime: hermes | claude-agent-sdk | deterministic
  status: succeeded | failed | needs_clarification | needs_approval
  output: object
  citations: SourceRef[]
  proposedActions: ProposedAction[]
  knowledgeMutationPlan?: KnowledgeMutationPlan
  logs: AgentLogEntry[]
}
```

13.2 Hermes Adapter Requirements

- Support bounded Hermes runs via the best validated surface: API server, TUI gateway, Python wrapper, or Kanban bridge.
- Support MCP tool access, but prefer routing mutating tools through the Tool Gateway.
- Support structured-output validation externally even if Hermes does not natively guarantee schema outputs.
- Support progress events, stop/cancel, and logs when available.
- Treat Hermes cron and Kanban as runtime capabilities, not the product workflow source of truth.
- Support workspace-scoped prompts and tool policies.

13.3 Claude Agent SDK Adapter Requirements

- Use TypeScript SDK in V1 to align with Tauri/React/TypeScript stack.
- Use JSON Schema structured outputs for workflow results.
- Use permission modes, tool allow/deny, canUseTool, hooks, and MCP server configuration.
- Support session persistence and external transcript mirroring where feasible.
- Support sandboxed execution via process override or containerization spike.
- Use as fallback or preferred runtime for high-structure jobs that require strict output schema.

13.4 Runtime Selection Policy

Job Type	Preferred Runtime	Rationale
Strict structured extraction	Claude Agent SDK	Native structured-output and permission surface.
Messaging-native interaction	Hermes	Already lives on messaging channels and supports gateway interactions.
MCP-heavy exploratory task	Hermes or Claude SDK	Select based on tool reliability and workspace policy.
Parallel research fanout	Hermes subagents/Kanban or Claude SDK subagents	Capability-specific spike decides default.
Deterministic parsing/progress	Deterministic worker	No LLM required for checkbox counts, hashes, IDs, or policy.

14. Knowledge System: Obsidian + GBrain + KnowledgeWriter

14.1 Knowledge Ownership

Markdown is canonical for semantic knowledge. GBrain is required for indexing, retrieval, graph, schema, synthesis, health, and internal jobs. The control plane stores workflow state, approvals, and audit, but not durable semantic knowledge.

14.2 KnowledgeWriter Requirements

- Accept only structured KnowledgeMutationPlans.
- Resolve note IDs and prevent duplicate pages.
- Validate workspace, source evidence, frontmatter, links, and generated-section ownership.
- Preserve user-owned sections.
- Perform three-way merge or create conflict review item when necessary.
- Write to Markdown atomically.
- Record revision, actor, source event, workflow run, and idempotency key.
- Trigger GBrain sync/index/dream jobs as appropriate.
- Expose failed write-through items in System Health until resolved.

14.3 Note Ownership Markers

```
## Current status

<!-- assistant:generated:project-status:start -->

Generated status goes here.

<!-- assistant:generated:project-status:end -->


## My notes

Human-owned content goes here. The assistant may read but not overwrite.
```

14.4 GBrain Capability Use in V1

GBrain Capability	V1 Use
Search/query/think	Used by all workflows to locate project, person, decision, meeting, source, and concept context.
Typed graph	Used to connect projects, people, meetings, decisions, sources, and workspaces.
Schema packs	Custom assistant schema pack defines page types/link types for the system.
MCP	Primary knowledge interface for Hermes and Claude SDK when appropriate.
Minions	Used for sync, embed, extract, dream, health, and other GBrain-internal jobs.
Dream cycle	Enabled through controlled bridge; durable semantic outputs must go through KnowledgeWriter.
Health/doctor	Displayed in System Health and used by maintenance workflows.
Ingestion contract	Evaluated and used where stable; adapter may bridge canonical ingestion events to GBrain ingestion.

15. Obsidian Second Brain Capability Coverage

The build should take the useful capabilities from Obsidian Second Brain, but not run the original repository as an independent writer. The useful parts become workflow semantics, source adapters, and skill specifications routed through the new control plane, GBrain, and KnowledgeWriter.

15.1 Capability Inventory and Disposition

OSB Capability	Disposition in This Build	Priority
/obsidian-world context loading	Port as Workspace/Global Context Loader using GBrain search and project registry.	P0
/obsidian-project	Port project creation/update semantics into Project Workflow.	P0
/obsidian-task	Port task triage/routing semantics into Task Router for Todoist/Linear/Asana.	P0
/obsidian-meeting	Port meeting preparation and meeting-note semantics.	P0
/obsidian-agenda / schedule / calendar	Port calendar agenda, conflict, focus-gap, attendee-context, and scheduling policies.	P0
/obsidian-save	Port as Session Checkpoint workflow. Hermes/Claude sessions submit durable knowledge candidates.	P0
/obsidian-ingest	Replace with canonical ingestion workflow; retain semantic extraction and routing ideas.	P0
/youtube	Port source-specific YouTube extractor as YouTubeAdapter.	P1
/podcast	Port RSS/transcript/Whisper fallback behavior as PodcastAdapter.	P1
/obsidian-review / recap / daily	Port daily close, weekly review, monthly review, and recap semantics.	P0/P1
Background agent	Replace with Temporal Session Checkpoint + GBrain dream/maintenance bridge.	P0
Four scheduled agents	Implement as product workflows: morning, nightly, weekly, health; add monthly.	P0/P1
/obsidian-architect	Port into Implementation Plan / GitHub / codebase progress workflow.	P1
/notebooklm	Replace/extend with NotebookLM sync workflow and API spike; retain vault-grounded synthesis concept.	P1
AI-first note rules	Incorporate into custom schema pack and KnowledgeWriter validations.	P0
Bi-temporal facts	Blend with GBrain timelines/takes and schema fields.	P1
Telegram catchup	Implement as Telegram capture + Ingestion Inbox workflow.	P1
Research/X/web/youtube/podcast workflows	Port high-value source adapters after meeting/project core flows.	P1/P2

15.2 Explicit Non-Adoption

- Do not allow original OSB slash commands to directly rewrite the Markdown brain in production.
- Do not run OSB background agent independently from GBrain dream or Temporal workflows.
- Do not keep separate OSB and GBrain knowledge stores.
- Do not duplicate task ownership between Obsidian notes and Todoist/Linear/Asana.
- Do not assume every source should deep-rewrite many existing notes; use ingestion profiles and policy.

16. Security, Privacy, and Safety Requirements

16.1 Threat Model

Threat	Mitigation
Prompt injection from transcripts, calendar descriptions, docs, web pages, NotebookLM outputs, or Markdown sources	Treat imported content as untrusted data; source-processing agents lack mutating tools; use prompt-injection scanning and policy gates.
Workspace leakage	Separate workspace credentials, GBrain brain/source scope, visibility levels, and synthetic adversarial tests.
Duplicate external writes on retry	External write envelope with idempotency key, canonical object key,

	preconditions, and write receipt.
Unapproved shared changes	Approval required for invites, shared meeting changes, assigning others, shared PM status changes, deletion, and external messages.
Secrets in Markdown	Secrets stored only in OS keychain/secret manager/env; scan notes for accidental secrets.
Agent runtime overreach	Runtimes receive tool policies; mutating tools route through Tool Gateway and KnowledgeWriter.
Corrupted knowledge writes	Atomic writes, revisions, diff/audit, retry outbox, conflict review, and GBrain parity checks.
Open-source install misconfiguration	Safe defaults, local-only binding, explicit token scopes, install doctor, health dashboard.

16.2 Autonomy Policy

Action	Default V1 Policy
Read notes, calendar, tasks, meetings, source docs	Automatic, workspace-scoped.
Write Markdown brain notes	Automatic if low-risk and evidence-backed, through KnowledgeWriter, with audit.
Create private personal task	Automatic if explicit user commitment or user request.
Create private focus block	Automatic if no invitees and no conflict.
Create self-assigned Linear/Asana task from explicit commitment	Automatic or low-friction approval depending workspace policy.
Invite other people to events	Approval required.
Assign task to someone else	Approval required.
Modify shared project status	Approval required.
Delete/archive notes or tasks	Approval required.
Send email/message externally	Denied in V1 unless a future feature enables explicit approval.
Resolve ambiguous contradictions	Approval required.

16.3 Audit Requirements

- Every workflow run has an ID, workspace, trigger, actor, and idempotency key.
- Every agent job records runtime, input context references, output schema, result, cost, and logs where available.
- Every KnowledgeWriter mutation records before/after diff or patch summary.
- Every external write records tool, target system, payload hash, approval ID if any, result, and external object ID.
- Every failed write or sync remains visible until resolved.

17. Non-Functional Requirements

Category	Requirement
Reliability	Meeting closeout, brief generation, and message intake survive worker restarts and transient connector failures without duplicate writes.
Performance	Dashboard loads from read models without requiring live LLM calls; normal views target sub-2-second load after local cache warmup.
Offline behavior	Mac app can read local Markdown/Obsidian state while cloud/control-plane availability is degraded; queued changes reconcile later.
Extensibility	All integrations and runtimes use adapter interfaces.
Portability	Knowledge remains understandable Markdown even if the app is removed.
Observability	Expose workflow status, queue depth, connector health, GBrain health, sync lag, cost, errors, and approval backlog.
Installability	Open-source users can install with documented local dev path and optional Docker Compose services.
Maintainability	Pin external runtimes and GBrain versions; include smoke tests and upgrade checklist.
Privacy	Workspace boundaries are enforced in runtime prompts, tool scopes, storage, and dashboard filters.

18. Implementation Plan and Milestones

18.1 Recommended Build Sequence

Phase	Scope	Exit Criteria
0. Architecture spikes	Hermes adapter path, Claude SDK adapter, GBrain round-trip, NotebookLM API, meeting closeout synthetic test.	Spike reports with go/no-go and constraints.
1. Foundation	Control plane skeleton, data model, workspace registry, project registry, event store, audit, KnowledgeMutationPlan schema.	Can register workspaces/projects and persist workflow events.
2. Knowledge substrate	Markdown repos, Obsidian-compatible structure, GBrain PGLite, GBrainAdapter, KnowledgeWriter prototype.	Obsidian edit -> GBrain search -> KnowledgeWriter update -> Obsidian valid Markdown.
3. Runtime adapters	HermesRuntimeAdapter and ClaudeAgentSdkRuntimeAdapter with structured result validation.	Both adapters can execute the same bounded test workflow.
4. Meeting closeout MVP	Granola + Calendar correlation + project/person context + tasks/actions + notes.	One real or synthetic meeting closes end-to-end without duplicates.
5. Task/calendar connectors	Todoist, Linear, Asana, Google Calendar write policies and approval UI.	External writes are idempotent and approval-gated.
6. Mac app MVP	Dashboard, Copilot, Ingestion Inbox, Approval Inbox, Project view, Calendar view.	User can operate the system from Mac UI.
7. Briefs/reviews	Morning brief, daily close, weekly review, monthly review, health workflow.	Briefs generated, saved to Markdown, and displayed.
8. Source adapters	YouTube, podcast, PDF/OCR, Telegram voice/files, GitHub/code, Drive docs.	Sources route correctly and update knowledge through KnowledgeWriter.
9. NotebookLM	API path if validated; otherwise Drive-managed source documents.	Project source pack syncs to NotebookLM path.
10. Open-source install	Docs, scripts, example config, sample workspace presets, local Docker Compose.	Fresh install from GitHub succeeds on clean environment.

18.2 V1 Scope

- Mac-first Tauri app from source or local build, direct signing optional.
- Three logical workspaces with global coordination metadata.
- Markdown Git repos opened in Obsidian.
- GBrain PGLite required; Postgres/Supabase migration path documented.
- Hermes and Claude Agent SDK runtime adapters.
- Temporal workflows for core cross-system flows.
- Granola, Google Calendar, Todoist, Linear, Asana, Telegram, Google Drive/Docs, GitHub.
- Meeting closeout, daily brief, project dashboard, task routing, calendar coordination, approvals.
- KnowledgeWriter and one-writer semantic write policy.
- GBrain full capability surface available through controlled layer.

18.3 V1.1 and Future Scope

Release	Candidate Features
V1.1	NotebookLM direct API if validated, richer Hermes Kanban, Postgres/Supabase migration, Gmail, Slack, advanced OCR/PDF, improved installer, project templates.
V1.2	Multi-machine sync hardening, mobile companion, more PM integrations, advanced GBrain schema packs, richer cross-workspace summaries.
Future	Optional multi-user/team mode, hosted deployment templates, plug-in marketplace, additional runtimes, enterprise security profiles.

19. Research Spikes and Open Questions

Spike	Question	Acceptance Criteria
Hermes Adapter Surface	Which Hermes surface is best for bounded structured jobs: API server, TUI gateway, Python wrapper, Kanban, or hybrid?	Run one meeting-close mock through Hermes with schema validation, stop/cancel, logs, and controlled tools.
Claude SDK Adapter	Can Claude Agent SDK run the same mock workflow with permissions, structured outputs, and MCP/GBrain context?	Adapter returns identical schema and obeys tool policy.
GBrain Round Trip	Can a Markdown repo opened in Obsidian remain canonical while GBrain indexes and controlled writes update it?	Edit in Obsidian, sync to GBrain, write via KnowledgeWriter, re-open in Obsidian, no malformed Markdown.
GBrain Full Capability Bridge	How should GBrain dream/Minions submit semantic outputs through KnowledgeWriter?	Dream job produces mutation plan or safe write bridge without DB-only semantic truth.
NotebookLM API	Is direct notebook/source CRUD/query/export possible through supported API?	Document official API feasibility; otherwise lock Drive-backed fallback.
Granola Connector	What is the best supported way to pull transcripts, metadata, summaries, decisions, and actions?	One transcript ingests with external ID and calendar correlation.
Cross-Workspace Policy	What metadata can safely coordinate across workspaces?	Adversarial tests prove personal/work leakage rules.
Implementation Progress	Can IMPLEMENTATION_PLAN.md plus Linear status produce robust technical project progress?	Parse real plan and cross-check with Linear without invented percentages.

19.1 Remaining Open Questions

- Will V1 ship with Temporal local dev server, Temporal Cloud, or both documented?
- Will GBrain run as a library/CLI subprocess, MCP service, or separate sidecar container in local install?
- How will user secrets be stored on Mac: Keychain, .env, 1Password CLI, or pluggable secret providers?
- What is the default retention policy for raw transcripts and audio-derived materials?
- How much of GBrain dream/autopilot should be enabled by default versus opt-in?
- Should personal-business be its own GBrain brain or a source inside a personal-owned brain at initial install?
- Should Markdown repos be one repo per workspace or a monorepo with workspace subdirectories?

20. Acceptance Criteria

20.1 End-to-End Acceptance Tests

Test	Pass Criteria
Meeting closeout replay	Same Granola event processed twice yields one meeting note, one set of tasks, one set of calendar proposals, one audit trail.
Workspace routing	Synthetic work, side-project, and personal-life meetings route to correct workspaces and project notes.
Cross-calendar scheduling	Doctor appointment workflow reads global busy/free and avoids work and side-project conflicts.
Knowledge write	Approved project update appears in Markdown, Obsidian, GBrain search, and dashboard within expected sync window.
Approval flow	Shared calendar invite proposal appears in Mac and Telegram, can be edited, approved, rejected, and audited.
Project progress	Implementation plan parser computes progress from checkbox evidence and displays next unchecked task.
Prompt injection	Transcript containing malicious instructions cannot make the system bypass policy or write to external systems.
Open-source install	A clean machine can run documented install and execute the sample workspace demo.

20.2 Definition of Done for V1

- The user can run the system locally/self-hosted with documented setup.
- Mac app shows global and workspace-specific dashboard data.
- Hermes and Claude SDK adapters can execute bounded jobs.
- GBrain is required, configured, and providing search/think/graph/health.
- Markdown repos can be opened in Obsidian and remain valid after system writes.
- Granola meeting closeout works end-to-end.
- Google Calendar, Todoist, Linear, and Asana writes are governed by policy and idempotency.
- Daily brief and weekly review are generated and saved.
- Approval inbox works from Mac and Telegram.
- At least one YouTube or podcast source adapter is operational or explicitly deferred with a ticket.

21. Risks and Mitigations

Risk	Impact	Mitigation
Over-complex architecture	Slow delivery and brittle integration.	Phase sequence starts with meeting closeout; defer non-essential connectors.
GBrain write-through divergence	Hidden DB truth or stale Markdown.	One-writer policy, parity checks, write outbox, GBrain read-heavy until validated.
Hermes integration instability	Runtime adapter unreliable.	Also ship Claude Agent SDK adapter; keep runtime broker pluggable.
Workspace leakage	Privacy/security failure.	Visibility levels, scoped prompts/tools, synthetic leakage tests.
Duplicate external writes	Bad calendar/task state.	Idempotency keys, write receipts, Temporal workflow state, precondition checks.
NotebookLM API unavailable	Direct sync impossible.	Drive-backed managed source docs fallback.
Too much autonomy too early	User trust loss.	Default sensitive writes to approval; expose clear audit and undo/correction paths.
Open-source install burden	Other users cannot adopt.	Presets, sample config, doctor command, local dev mode, docs.
Cost explosion	High API bills.	Runtime budgets, maxCostUsd, job caps, GBrain local PGLite, cache, deterministic first.
Prompt injection	Unsafe actions or corrupted notes.	Untrusted source handling, tool gateway, approval policy, sandboxing, red-team tests.

22. References

The PRD is based on user-provided deep research results, direct repository review, and public project/vendor documentation. The following references should be revisited during implementation spikes.

ID	Source	URL
R1	Hermes Agent README	https://github.com/NousResearch/hermes-agent
R2	Hermes Programmatic Integration Docs	https://hermes-agent.nousresearch.com/docs/developer-guide/programmatic-integration
R3	Hermes MCP Feature Docs	https://hermes-agent.nousresearch.com/docs/user-guide/features/mcp
R4	Hermes Cron Feature Docs	https://hermes-

		agent.nousresearch.com/docs/user-guide/features/cron
R5	Hermes Delegation/Subagents Docs	https://hermes-agent.nousresearch.com/docs/user-guide/features/delegation
R6	Hermes Kanban Feature Docs	https://hermes-agent.nousresearch.com/docs/user-guide/features/kanban
R7	Claude Agent SDK Overview	https://docs.anthropic.com/en/docs/claude-code/sdk/sdk-overview
R8	Claude Agent SDK TypeScript Reference	https://docs.anthropic.com/en/docs/claude-code/sdk/typescript
R9	Claude Agent SDK Permissions and Tool Use	https://docs.anthropic.com/en/docs/claude-code/sdk/permissions
R10	Claude Agent SDK Hooks	https://docs.anthropic.com/en/docs/claude-code/sdk/hooks
R11	GBrain Repository	https://github.com/garrytan/gbrain
R12	GBrain Retrieval Architecture	https://github.com/garrytan/gbrain/blob/main/docs/architecture/RETRIEVAL.md
R13	GBrain Engines Documentation	https://github.com/garrytan/gbrain/blob/main/docs/ENGINES.md
R14	GBrain Brains and Sources	https://github.com/garrytan/gbrain/blob/main/docs/architecture/brains-and-sources.md
R15	GBrain Minions Deployment Guide	https://github.com/garrytan/gbrain/blob/main/docs/guides/minions-deployment.md
R16	GBrain Cron Schedule Guide	https://github.com/garrytan/gbrain/blob/main/docs/guides/cron-schedule.md
R17	GBrain Security Notes	https://github.com/garrytan/gbrain/blob/main/SECURITY.md
R18	Obsidian Second Brain Repository	https://github.com/eugeniughelbur/obsidian-second-brain
R19	NotebookLM Sources Help	https://support.google.com/notebooklm/answer/16215270
R20	NotebookLM Notebook and Artifact Help	https://support.google.com/notebooklm/answer/16206563
R21	Temporal Workflow Concepts	https://docs.temporal.io/workflows
R22	Telegram Bot API	https://core.telegram.org/bots/

		api
R23	Asana MCP Server Docs	https://developers.asana.com/docs/using-asanas-mcp-server
R24	Linear MCP Docs	https://linear.app/docs/mcp
R25	Todoist Developer Docs	https://developer.todoist.com/rest/v2/
R26	Granola MCP Docs	https://docs.granola.ai/help-center/sharing/integrations/mcp

22.1 Notes on Evidence Level

- Hermes and Claude Agent SDK capabilities should be revalidated against pinned versions during implementation.
- GBrain Minions, dream, ingestion, and schema-pack internals remain required V1 capabilities but need integration spikes before being relied on for production writes.
- NotebookLM direct API remains unconfirmed; Drive-backed managed sources are the default fallback.
- Vendor APIs for Granola, Calendar, Todoist, Linear, Asana, Drive, and Telegram should be checked during connector implementation for current rate limits, scopes, and endpoint behavior.